2.9 signal.h

The signal header provides a means to handle signals reported during a program's execution.

Macros:

```
SIG_DFL
SIG_ERR
SIG_IGN
SIGABRT
SIGFPE
SIGILL
SIGINT
SIGSEGV
SIGTERM
```

Functions:

```
signal();
raise();
```

Variables:

```
typedef sig_atomic_t
```

2.9.1 Variables and Definitions

The `sig_atomic_t` type is of type `int` and is used as a variable in a signal handler. The `SIG_` macros are used with the signal function to define signal functions.

`SIG_DFL` Default handler.

`SIG_ERR` Represents a signal error.

`SIG_IGN` Signal ignore.

The `SIG` macros are used to represent a signal number in the following conditions:

`SIGABRT` Abnormal termination (generated by the abort function).

`SIGFPE` Floating-point error (error caused by division by zero, invalid operation, etc.).

`SIGILL` Illegal operation (instruction).

`SIGINT` Interactive attention signal (such as ctrl-C).

`SIGSEGV` Invalid access to storage (segment violation, memory violation).

`SIGTERM` Termination request.

2.9.2 signal

Declaration:

```
void (*signal(int sig, void (*func)(int)))(int);
```

Controls how a signal is handled. *sig* represents the signal number compatible with the `SIG` macros. *func* is the function to be called when the signal occurs. If func is `SIG_DFL`, then the default handler is called. If *func* is `SIG_IGN`, then the signal is ignored. If *func* points to a function, then when a signal is detected the default function is called (`SIG_DFL`), then the function is called. The function must take one argument of type `int` which represents the signal number. The function may terminate with `return`, `abort`, `exit`, or `longjmp`. When the function terminates execution resumes where it was interrupted (unless it was a `SIGFPE` signal in which case the result is undefined).

If the call to signal is successful, then it returns a pointer to the previous signal handler for the specified signal type. If the call fails, then `SIG_ERR` is returned and `errno` is set appropriately.

2.9.3 raise

Declaration

```
int raise(int sig);
```

Causes signal *sig* to be generated. The *sig* argument is compatible with the `SIG` macros.

If the call is successful, zero is returned. Otherwise a nonzero value is returned.

Example:

```
#include<signal.h>
#include<stdio.h>

void catch_function(int);

int main(void)
{
  if(signal(SIGINT, catch_function)==SIG_ERR)
   {
    printf("An error occured while setting a signal handler.\n");
    exit(0);
   }

  printf("Raising the interactive attention signal.\n");
  if(raise(SIGINT)!=0)
   {
    printf("Error raising the signal.\n");
    exit(0);
   }
  printf("Exiting.\n");
  return 0;
}

void catch_function(int signal)
{
  printf("Interactive attention signal caught.\n");
}
```

The output from the program should be (assuming no errors):

```
Raising the interactive attention signal.
```

```
Interactive attention signal caught.
Exiting.
```